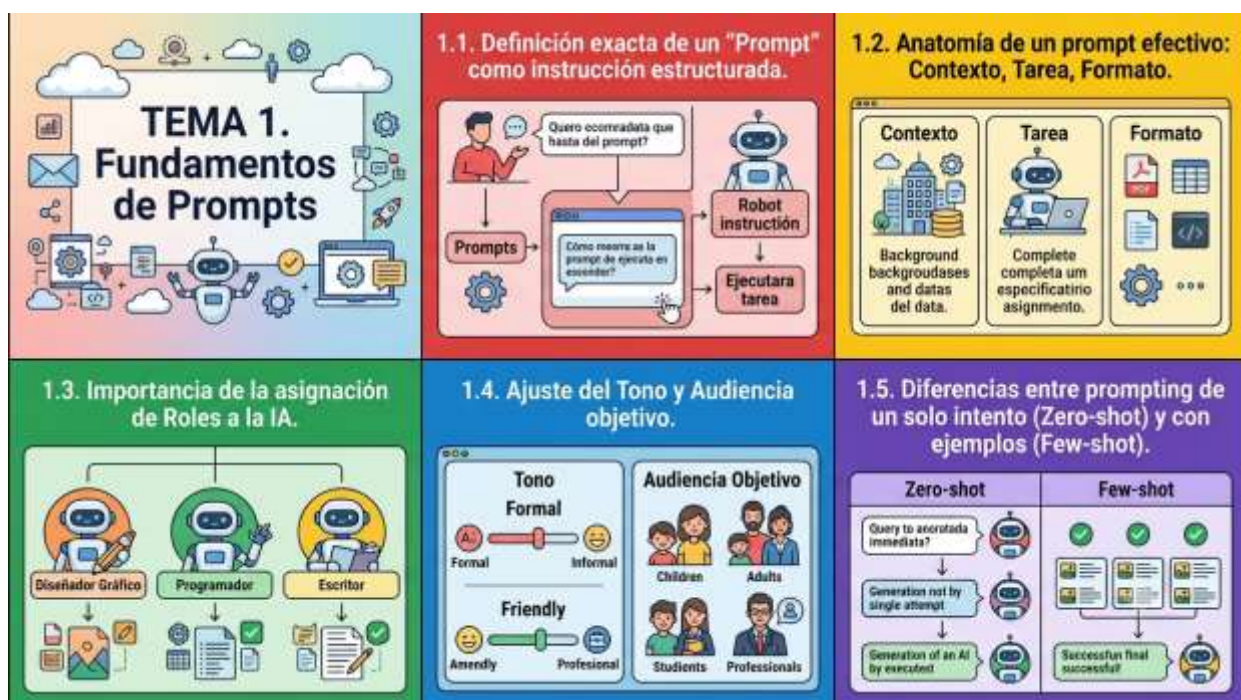


TEMA N° 1



TEMA 1. FUNDAMENTOS DE PROMPTS



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender la naturaleza y estructura de un prompt**, identificándolo como una instrucción lógica y precisa dentro del desarrollo de software y la interacción con modelos de Inteligencia Artificial.
2. **Diseñar e implementar prompts de alta efectividad** aplicando componentes clave como contexto, tarea y formato, adaptados a resolver necesidades reales del entorno productivo y comercial.
3. **Dominar las metodologías de interacción avanzadas (Zero-shot y Few-shot)**, dotando al futuro Técnico de herramientas críticas para optimizar la automatización de procesos informáticos y la auditoría de datos locales.



PRÁCTICA



PREGUNTA DETONANTE

Si las computadoras siguen instrucciones lógicas escritas en lenguajes de programación, ¿por qué los nuevos modelos de Inteligencia Artificial parecen entender nuestro propio lenguaje, y qué pasa cuando no sabemos darles la orden exacta?

EL PROBLEMA TECNOLÓGICO EN NUESTRO ENTORNO

Imaginemos la siguiente situación en el municipio de Pailón. Don Jorge, un conocido comerciante del **Barrio Central Fátima**, posee un almacén de abarrotes y productos agropecuarios de alta demanda. Con la reciente entrega del nuevo módulo educativo del **CEA Herman Ortiz Camargo** en el **Barrio Saó**, un grupo de estudiantes de Segundo Semestre de Sistemas Informáticos decidió diseñar un sistema informático base para ayudar a Don Jorge a clasificar automáticamente las solicitudes de pedidos que le llegan por mensajes de texto desde comunidades alejadas y de barrios como **María Belén, San Antonio y Jardines de Pailón**.

El estudiante a cargo del módulo de Inteligencia Artificial decidió usar un modelo de lenguaje para leer los mensajes entrantes y extraer de forma automática tres datos: Producto, Cantidad y Barrio de destino.

Sin embargo, al ingresar el primer mensaje recibido desde el **Barrio 24 de Septiembre** (cerca de la estación de tren):

"Hola Don Jorge, por favor me lo alista tres bolsas de alimento para pollos y dos de maíz, voy a pasar a recogerlo en mototaxi después de jugar en los campos deportivos de la 24 de Septiembre, me lo manda la nota de venta por favor".

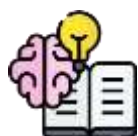
El estudiante simplemente escribió al modelo de IA:

"Revisa este mensaje y dime qué quieren".



El resultado de la IA fue un párrafo largo, desorganizado, que olvidó mencionar la cantidad exacta de maíz y confundió el lugar de entrega con el lugar de origen. Don Jorge no pudo usar la información para preparar el pedido a tiempo, generando retrasos en la atención.

Este problema real nos demuestra que el modelo de IA no falló por falta de capacidad, sino porque la instrucción carecía de estructura, límites y especificaciones técnicas. Como Técnicos en Sistemas Informáticos, nuestra labor es construir el puente perfecto entre las necesidades del usuario y la respuesta de la máquina.



TEORÍA



1.1. DEFINICIÓN EXACTA DE UN "PROMPT" COMO INSTRUCCIÓN ESTRUCTURADA

En el ámbito de la informática tradicional, un programa se ejecuta mediante líneas de código imperativas o declarativas dentro de un entorno controlado. En el paradigma de la Inteligencia Artificial Generativa y los Modelos de Lenguaje de Gran Escala (LLMs), un **prompt** es el punto de entrada de datos (input) constituido por texto en lenguaje natural que actúa como una instrucción estructurada, diseñada específicamente para guiar, condicionar y limitar el comportamiento de un modelo probabilístico.

Un prompt no es un simple saludo o una frase al azar; es una interfaz de programación en lenguaje humano. El modelo de IA funciona prediciendo la palabra más probable que debe seguir al texto que ha recibido. Por lo tanto, si la instrucción carece de una arquitectura lógica, el modelo dispersará su atención en su gigantesca base de datos, produciendo respuestas genéricas o erróneas. Al estructurar un prompt, el Técnico define un espacio de búsqueda específico, reduciendo la incertidumbre del modelo y asegurando un resultado de alta precisión y repetibilidad técnica.

1.2. ANATOMÍA DE UN PROMPT EFECTIVO: CONTEXTO, TAREA, FORMATO

Para garantizar que un prompt devuelva información útil en proyectos de desarrollo de software o gestión de sistemas, debemos diseñarlo utilizando una anatomía tripartita obligatoria.



1. **Contexto:** Establece los antecedentes, la situación actual, las restricciones y el entorno en el que se enmarca el problema. Define el "por qué" y el "dónde".
2. **Tarea:** Es la acción directa, clara e inequívoca que el modelo debe ejecutar. Se recomienda iniciar siempre con un verbo de acción en imperativo (Extrae, Clasifica, Traduce, Genera, Modifica).
3. **Formato:** Determina la estructura de salida que debe tener la respuesta. Puede ser un archivo JSON, una tabla Markdown, una lista de viñetas, código fuente limpio o un párrafo de extensión limitada.



Si aplicamos esta anatomía al problema del comerciante en el **Barrio Central Fátima**, el prompt estructurado se diseñaría de la siguiente forma dentro de nuestro entorno de desarrollo:

Markdown

CONTEXTO:Eres un asistente informático especializado en la gestión de inventarios y logística de distribución para comercios en el municipio de Pailón. Procesas mensajes de clientes locales para organizar rutas de entrega.TAREA:Analiza el siguiente mensaje de texto entrante. Extrae estrictamente los siguientes campos:1. Nombre del producto.2. Cantidad solicitada (en números).3. Barrio de destino de Pailón (identifica si pertenece a Barrio Central Fátima, 24 de Septiembre, Residencial Norte, 27 de Mayo, Saó u otros).FORMATO:Devuelve los datos exclusivamente en una tabla Markdown con las



columnas: [Producto | Cantidad | Barrio_Destino]. No agregues comentarios adicionales ni introducciones. TEXTO A ANALIZAR: "

Hola Don Jorge, por favor me lo alista tres bolsas de alimento para pollos y dos de maíz, voy a pasar a recogerlo en mototaxi después de jugar en los campos deportivos de la 24 de Septiembre, me lo manda la nota de venta por favor"

1.3. IMPORTANCIA DE LA ASIGNACIÓN DE ROLES A LA IA

La asignación de un rol dentro de un prompt consiste en instruir al modelo para que adopte una personalidad profesional, técnica o conceptual específica antes de procesar la tarea. En términos de arquitectura de IA, esto modifica los pesos de atención internos del modelo, priorizando el vocabulario, las estructuras sintácticas y las metodologías de resolución de problemas propias del campo asignado.

No es lo mismo solicitar código fuente a una IA genérica que asignarle el rol de: "*Actúa como un Administrador de Bases de Datos senior experto en PostgreSQL*". En el segundo caso, el modelo omitirá explicaciones superficiales, estructurará las consultas utilizando buenas prácticas de indexación, normalización y seguridad, y preverá posibles fallas de concurrencia. Para un Técnico, dominar la asignación de roles permite subcontratar lógicamente tareas de alta complejidad, obteniendo respuestas con un estándar profesional verificado.

1.4. AJUSTE DEL TONO Y AUDIENCIA OBJETIVO

El **tono** define la actitud y el estilo de la comunicación (formal, técnico, persuasivo, descriptivo, empático), mientras que la **audiencia** determina a quién va dirigida la información resultante. Al desarrollar soluciones de software, la flexibilidad en el manejo de estos dos parámetros es vital.

Por ejemplo, si el coliseo municipal ubicado en el **Barrio 27 de Mayo** requiere un reporte automatizado sobre fallas en el cableado de red de sus oficinas de administración, el Técnico puede configurar el prompt para ajustar el resultado según el destinatario:

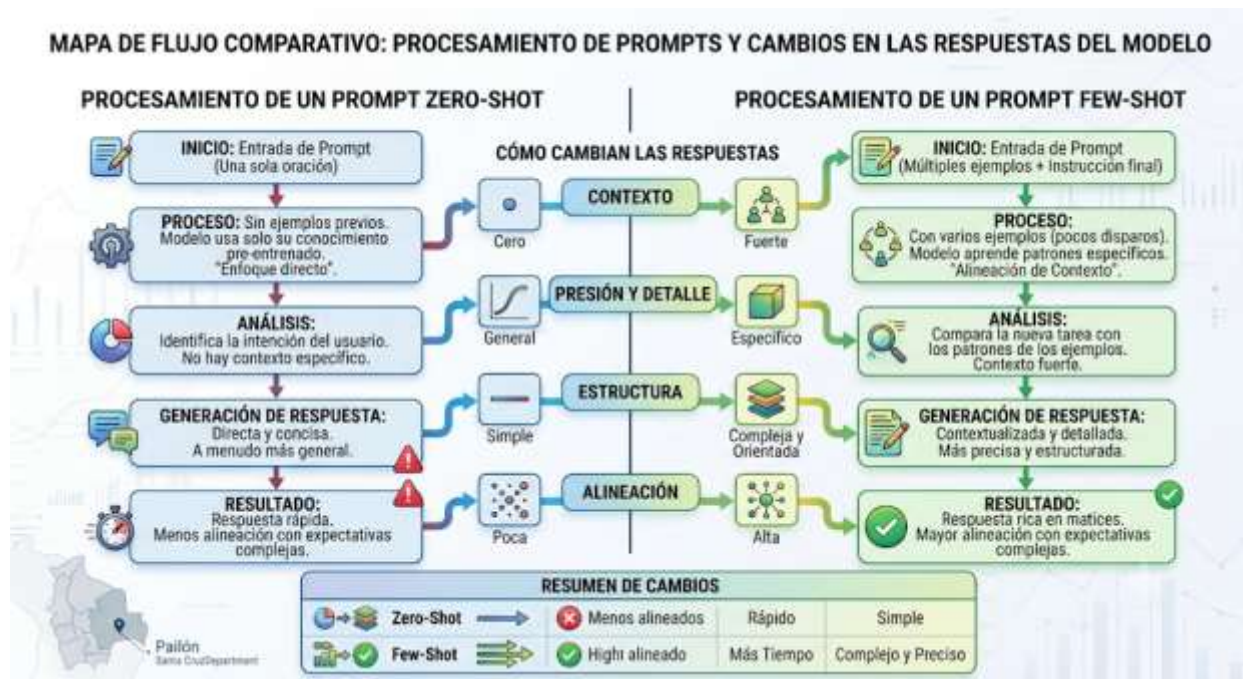
1. **Audiencia - Alcaldía Municipal (Autoridades):** Tono formal, ejecutivo, enfocado en costos, plazos de ejecución y beneficios institucionales, omitiendo tecnicismos complejos.
2. **Audiencia - Soporte Técnico (Compañeros Informáticos):** Tono estrictamente técnico, descriptivo, enfocado en protocolos de red, topologías, categorías de cables (Cat6) y configuraciones de switches.



1.5. DIFERENCIAS ENTRE PROMPTING DE UN SOLO INTENTO (ZERO-SHOT) Y CON EJEMPLOS (FEW-SHOT)

La capacidad de un modelo de IA para resolver una tarea depende directamente de cómo le presentemos la información de referencia en el prompt.

1. **Prompting de un solo intento (Zero-shot):** Consiste en solicitar al modelo que realice una tarea sin proporcionarle ningún ejemplo previo de cómo queremos que sea el resultado. Se apoya completamente en el entrenamiento base del modelo. Es ideal para tareas comunes de clasificación o traducción simple.
2. **Prompting con ejemplos (Few-shot):** Consiste en incluir dentro del prompt uno o más ejemplos claros de entradas con sus respectivas salidas correctas antes de presentar la entrada real que queremos procesar. Es indispensable cuando requerimos que la IA aprenda un formato personalizado, una lógica interna de nuestro sistema, o reglas de negocio muy específicas que no forman parte del conocimiento general de internet.



A continuación, se presenta un diseño técnico de prompt en modalidad **Few-shot** para estandarizar la clasificación de reportes de mantenimiento del hardware del nuevo módulo del **CEA Herman Ortiz Camargo**:



Markdown

Eres un sistema automatizado de triaje de soporte técnico para laboratorios de computación en Pailón. Tu objetivo es clasificar el componente afectado y el nivel de prioridad. EJEMPLO 1: Usuario: "

La computadora 5 del laboratorio del Barrio Saó no enciende su pantalla, se queda en negro y parpadea un foco naranja."

Salida: [Componente: Monitor] [Prioridad: Media] EJEMPLO 2: Usuario: "

Hubo un corte por el Barrio Residencial Norte y ahora el servidor principal de la carrera no arranca, huele a quemado por la fuente."

Salida: [Componente: Servidor / Fuente de Poder] [Prioridad: Alta] EJEMPLO

3: Usuario: "

El mouse de la máquina del profesor se traba, el puntero salta de un lado a otro cuando estamos en clases."

Salida: [Componente: Periférico / Mouse] [Prioridad: Baja] TAREA REAL A

PROCESAR: Usuario: "

En el bloque nuevo del CEA, tres teclados no marcan las letras de la fila central debido al polvo acumulado por los vientos de la zona."

Salida:

1.6. TÉCNICAS AVANZADAS DE ENCADENAMIENTO DE PENSAMIENTOS (CHAIN-OF-THOUGHT) EN LA RESOLUCIÓN DE PROBLEMAS LÓGICOS

El encadenamiento de pensamientos u ontología de razonamiento secuencial (**Chain-of-Thought**) es una técnica avanzada de prompting que fuerza al modelo de lenguaje a descomponer un problema complejo en pasos lógicos intermedios antes de emitir la respuesta final. En lugar de pasar directamente del input al output, el modelo escribe explícitamente su proceso de razonamiento.





Esto es crítico para los Técnicos en Sistemas Informáticos al momento de depurar algoritmos o buscar fallas de conectividad en redes inalámbricas desplegadas en barrios extensos como **Ovidio Barberi** o **Las Misiones**. Al indicarle al modelo "*Piensa paso a paso*", aumentamos drásticamente la precisión del código generado, ya que cada línea de código producida se basará en la premisa lógica validada en el paso anterior, reduciendo errores sintácticos y lógicos.

1.7. MITIGACIÓN DE ALUCINACIONES EN MODELOS DE LENGUAJE PARA AUDITORÍA DE CÓDIGO

Las **alucinaciones** son fenómenos probabilísticos donde los modelos de IA generan afirmaciones falsas, datos inexistentes o funciones de programación que no pertenecen a ninguna librería real, presentándolas con un tono de total seguridad. En la auditoría de sistemas e infraestructura informática, una alucinación puede provocar fallos críticos de seguridad en servidores de producción.

Para mitigar las alucinaciones, el Técnico debe aplicar técnicas de **confinamiento de contexto** en sus prompts. Estas incluyen:

1. Declarar explícitamente la prohibición de inventar información ("*Si no encuentras la respuesta en el texto provisto, responde estrictamente: 'Información no disponible'*").
2. Forzar el anclaje a fuentes o documentación oficial proporcionada directamente en el cuerpo del prompt.
3. Exigir la cita de las líneas de código exactas o de los parámetros de red reales antes de proponer una corrección técnica.

1.8. AUTOMATIZACIÓN Y OPTIMIZACIÓN DE FLUJOS DE TRABAJO LOCALES MEDIANTE APIS DE IA

El verdadero valor de un prompt estructurado se alcanza cuando se integra dentro de una aplicación informática real mediante llamadas a Interfaces de Programación de Aplicaciones (APIs). Los Técnicos de Pailón no solo consumen IA desde interfaces web (como ChatGPT o Gemini); diseñan scripts capaces de inyectar variables del contexto local de forma dinámica en plantillas de prompts optimizadas.



A continuación, se expone un ejemplo de integración real utilizando lenguaje de programación Python. Este script simula la automatización de notificaciones para cortes de servicio de red que afectan a los usuarios del **Barrio San Expedito** y **Barrio Las Misiones**, enviando la información estructurada directamente a una base de datos o sistema de mensajería:

Python

```
# Script de automatización para generación de comunicados técnicos en Pailón
# Módulo: Sistemas Informáticos - CEA Herman Ortiz Camargodef
generar_prompt_comunicado(barrio_afectado, hora_corte, tiempo_estimado): #
Definición de la plantilla estructurada del prompt utilizando f-strings
prompt_base = f"""    CONTEXTO:    Eres el Administrador de Infraestructura de
Red de la cooperativa de servicios tecnológicos en Pailón.    Se ha detectado
una degradación en la fibra óptica debido a trabajos de pavimentación urbana.
TAREA:    Redacta un comunicado oficial de alerta para Los usuarios de la zona
afectada. El mensaje debe ser claro,    preventivo y empático con La población
afectada.    RESTRICCIONES GEOGRÁFICAS Y TÉCNICAS:    - Barrio afectado actual:
{
barrio_afectado}
- Hora de inicio del incidente: {
hora_corte}
- Tiempo de resolución estimado: {
tiempo_estimado}
horas    FORMATO:    Un único párrafo de máximo 60 palabras, optimizado para ser
enviado por SMS o canales de mensajería vecinal.    """    return prompt_base
# Simulación de variables capturadas dinámicamente desde el sistema de monitoreo
localbarrio = "
Barrio San Expedito" # Variable geográfica localhora = "14:30 PM"    #
Variable temporal del incidenteduracion = "2"    # Estimación de
trabajo técnico en sitio
# Generación del prompt final listo para ser enviado al modelo de IA mediante
APIprompt_listo = generar_prompt_comunicado(barrio, hora, duracion)
# Impresión del resultado en la consola del sistema informáticoprint("
--- PROMPT ESTRUCTURADO GENERADO PARA LA API -
--")
print(prompt_listo)
```

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito	Realidad
------	----------



La Inteligencia Artificial comprende el contexto del problema de la misma forma que un ser humano inteligente.	La IA no comprende significados; calcula probabilidades de aparición de palabras basándose en las instrucciones exactas provistas en el prompt.
Para obtener un buen código informático o un reporte técnico, basta con escribir prompts muy largos y detallados.	La longitud no garantiza precisión. Un prompt corto con un rol claro, restricciones estrictas y un formato definido es más efectivo que un texto extenso y redundante.

□ Glosario de Términos

1. **Prompt:** Instrucción o conjunto de condiciones en lenguaje natural provisto a un modelo de IA para dirigir su comportamiento y la estructura de su salida.
2. **Zero-shot:** Técnica de interacción donde se solicita una resolución o generación de contenido sin proveer ningún ejemplo previo al modelo.
3. **Few-shot:** Técnica de prompting que incluye ejemplos de pares (entrada/salida) para guiar al modelo en la adopción de formatos o reglas lógicas personalizadas.
4. **Alucinación:** Error de generación donde un modelo de lenguaje produce información falsa, inexacta o inventada con apariencia de coherencia técnica.
5. **Token:** Unidad básica de procesamiento de texto utilizada por los modelos de IA, equivalente aproximadamente a una palabra o fragmento de sílaba.

□ Ponte a Prueba

1. ¿Cuáles son los tres componentes esenciales de la anatomía de un prompt efectivo para la resolución de problemas técnicos?
1. A) Introducción, Desarrollo, Conclusión.
 2. B) Contexto, Tarea, Formato.
 3. C) Rol, Código, Longitud.
 4. D) Variables, Funciones, Comentarios.



2. Si necesitamos que un modelo de IA clasifique correos de reclamos vecinales del Barrio 27 de Mayo siguiendo una estructura de etiquetas personalizada que el modelo no conoce de antemano, ¿qué técnica debemos usar?

1. A) Prompting Zero-shot.
2. B) Prompting de texto plano libre.
3. C) Prompting Few-shot (con ejemplos).
4. D) Eliminación de tokens del sistema.

3. El fenómeno probabilístico en el cual un modelo de Inteligencia Artificial genera una función de código que no existe o datos geográficos falsos sobre Pailón se denomina:

1. A) Desbordamiento de búfer.
2. B) Alucinación.
3. C) Encadenamiento lógico.
4. D) Tokenización inversa.

Respuestas correctas: 1: B, 2: C, 3: B.



VALORACIÓN



El despliegue masivo de tecnologías basadas en Inteligencia Artificial y la optimización mediante ingeniería de prompts conllevan una profunda responsabilidad ética, social y medioambiental que todo Técnico en Sistemas Informáticos debe sopesar rigurosamente.

Desde una perspectiva **medioambiental**, el procesamiento de millones de prompts complejos a nivel global demanda una potencia de cómputo colosal en Centros de Datos internacionales. Esto se traduce en un consumo energético crítico y en la necesidad de sistemas de refrigeración masivos que evaporan millones de litros de agua dulce. Además, la rápida obsolescencia de los servidores de procesamiento acelera la acumulación de basura electrónica (*e-waste*), impactando directamente en ecosistemas globales. Por ende, optimizar nuestros prompts no es solo una



necesidad técnica de velocidad, sino un acto de eficiencia energética: un prompt mal estructurado requiere múltiples intentos repetitivos, duplicando innecesariamente la huella de carbono digital de nuestra consulta.

En el plano **social y ético**, el uso de IA para automatizar la redacción de informes, la toma de decisiones o la atención al cliente en municipios en crecimiento como Pailón plantea dilemas sobre la privacidad de los datos. Si un Técnico introduce datos sensibles o privados de los ciudadanos de los barrios de Pailón (como registros de deudas, nombres completos o historiales médicos) en prompts dirigidos a modelos comerciales en la nube, está violando principios básicos de confidencialidad y exponiendo de forma irreversible esa información a fugas de datos masivas. La Inteligencia Artificial debe ser vista como una herramienta de amplificación del trabajo del Técnico local, resguardando en todo momento la integridad y seguridad de nuestra comunidad.



PRODUCCIÓN



RETO FINAL: DISEÑO DE UN OPTIMIZADOR DE BASE DE DATOS PARA EL COMERCIO DE PAILÓN

Objetivo del Laboratorio: Aplicar las técnicas de asignación de roles, anatomía de prompts, prompting Few-shot y Chain-of-Thought para construir una plantilla de prompt profesional que audite e identifique errores lógicos en un script SQL destinado a gestionar las ventas del Barrio Central Fátima.

Pasos a seguir en la computadora:

1. Inicie su entorno de edición de texto y copie el siguiente fragmento de código SQL erróneo, simulando una base de datos local expuesta a fallas:

SQL

```
-- Script de Base de Datos para Control de Ventas - Pailón Comercio
-- Error intencional: Falta la llave primaria y la relación de llaves
foráneasCREATE TABLE ventas_barrio_fatima (    id_venta INT, -- Falta
```



```
AUTO_INCREMENT o PRIMARY KEY    producto_nombre VARCHAR(255),    barrio_origen
VARCHAR(100),    monto_total DECIMAL(10,2));
INSERT INTO ventas_barrio_fatima VALUES (1, '
Alimento Balanceado', '
Barrio Central Fátima', 350.00);
INSERT INTO ventas_barrio_fatima VALUES (1, '
Maíz Seleccionado', '
Barrio Oto Waver', 120.00);
-- Llave duplicada que el sistema no bloqueará
```

1. Abra la interfaz del modelo de lenguaje disponible en su laboratorio de computación del **CEA Herman Ortiz Camargo**.
2. Construya y ejecute un prompt estructurado aplicando estrictamente la siguiente plantilla de diseño técnico:

Markdown

ROL:Actúa como un Administrador de Bases de Datos (DBA) senior y auditor de seguridad de código para sistemas de gestión relacional MySQL / PostgreSQL.CONTEXTO:Este script se ejecutará en los servidores del laboratorio del Barrio Saó para procesar los datos consolidados de las ventas hechas en el Barrio Central Fátima y distribuidas a barrios aledaños como Barrio Oto Waver en Pailón. Necesitamos garantizar la integridad de los datos de los comerciantes.TAREA:Aplica la técnica de Chain-of-Thought (Pensamiento paso a paso) para:1. Analizar el código SQL provisto e identificar errores críticos de diseño de bases de datos (llaves primarias ausentes, falta de control de duplicados).2. Explicar detalladamente por qué estos errores afectarán el negocio de Don Jorge en Pailón si la base de datos crece.3. Generar el código SQL corregido, normalizado y con comentarios detallados en cada línea modificada.FORMATO DE SALIDA:Devuelve primero el análisis paso a paso en una lista numerada, y posteriormente el bloque de código SQL corregido dentro de un contenedor estructurado. Omite comentarios introductorios informales.

1. Verifique la salida generada por el modelo de IA. El resultado final debió reestructurar la tabla añadiendo campos como PRIMARY KEY, tipos de datos correctos y restricciones de integridad de datos, entregando una solución técnica válida para la implementación en el sistema informático del comercio de Pailón.

RECURSOS ADICIONALES

1. **Guía Oficial de Prompt Engineering de OpenAI:** Documentación técnica esencial que detalla las mejores prácticas internacionales para la estructuración de instrucciones y



control de formatos de salida.*Enlace recomendado:*
<https://platform.openai.com/docs/guides/prompt-engineering>

2. **Repositorio de Prompts Académicos de Anthropic:** Compendio técnico de patrones de diseño de prompts avanzados, asignación de roles complejos y control de alucinaciones en modelos de lenguaje de gran escala.
Enlace recomendado: <https://docs.anthropic.com/en/docs/prompt-library>
3. **Guía del Desarrollador sobre Inteligencia Artificial - GitHub Learn:** Manual de integración de variables dinámicas, prompts en código fuente Python y consumo eficiente de APIs de IA para Técnicos de software.
Enlace recomendado: <https://github.com/features/preview/copilot>